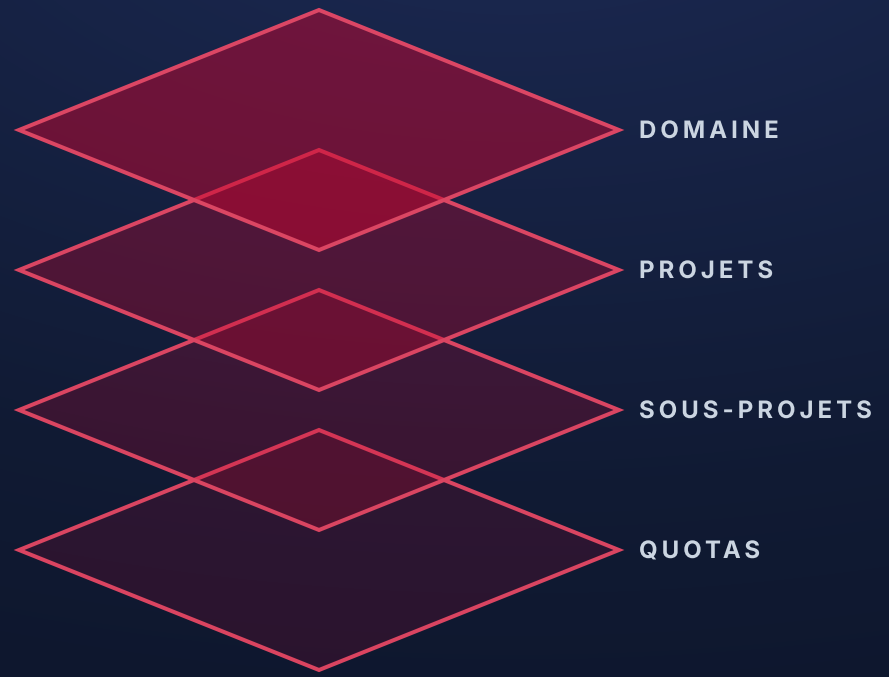
COMPRENDRE OPENSTACK · MODULE 10

Projets, hiérarchie et quotas

Organiser un système d'information cloisonné sur Canonical OpenStack (Sunbeam) : le projet comme unité de propriété, l'arbre de projets, les plafonds de consommation

Projet | Sous-projets | Rôles hérités | Quotas | Limites unifiées | Cloisonnement



Version déployée : Canonical OpenStack 2026.1 LTS, donc OpenStack 2026.1 (les modules 1 à 5 suivent l'amont 2026.2) · documentations consultées en octobre 2026

INTRODUCTION

Objectif et déroulé

Objectif : savoir découper un cloud en projets, les ranger dans une hiérarchie, borner leur consommation par des quotas, et choisir un modèle d'organisation qui cloisonne réellement équipes, applications et environnements.

À l'issue de ce module, vous saurez :

- Dire ce qu'un **projet** isole, service par service, et ce qu'il laisse partagé.
- Construire un **arbre de projets** et y poser des **rôles hérités**.
- Lire, fixer et dimensionner des **quotas** pour le calcul, le stockage et le réseau.
- Expliquer les **limites unifiées** et leurs deux modèles d'application.
- Comparer plusieurs **modèles d'organisation** et en choisir un.
- Compléter le cloisonnement par le réseau et le matériel.

1 · Le projet
Unité de propriété, isolation, partage, cycle de vie.

2 · La hiérarchie de projets
Arbre, règles, rôles hérités, ce qu'elle ne fait pas.

3 · Les quotas
Par service, vérification, lecture et réglage, limites unifiées, dimensionnement.

4 · Organiser un SI cloisonné
Niveaux de cloisonnement, quatre modèles, réseau et matériel.

5 · Mise en œuvre
Pas à pas sur Sunbeam, revue, bonnes pratiques.

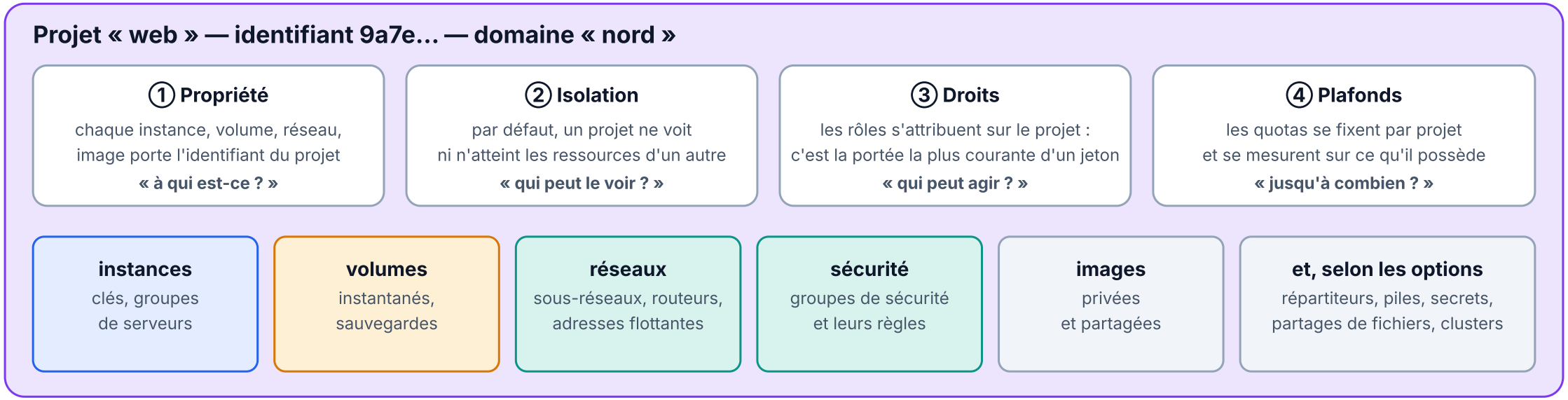
01

Le projet

Dans OpenStack, toute ressource appartient à un projet et à un seul. C'est l'unité sur laquelle reposent la propriété, l'isolation, les droits et les plafonds.

- Ce qu'est un projet
- Ce qui est étanche, ce qui est partagé
- Cycle de vie
- Les projets d'un cloud Sunbeam
- Ce qu'il porte, service par service
- Partager entre projets, volontairement
- Étiquettes et propriétés

Ce qu'est un projet



Vocabulaire : « projet » et « tenant » désignent la même chose ; le second est l'ancien nom, encore présent dans certaines commandes, options et messages d'erreur.

Un projet n'est pas un groupe de personnes. C'est un conteneur de ressources ; les personnes y accèdent par des rôles, et une même personne peut travailler dans plusieurs projets.

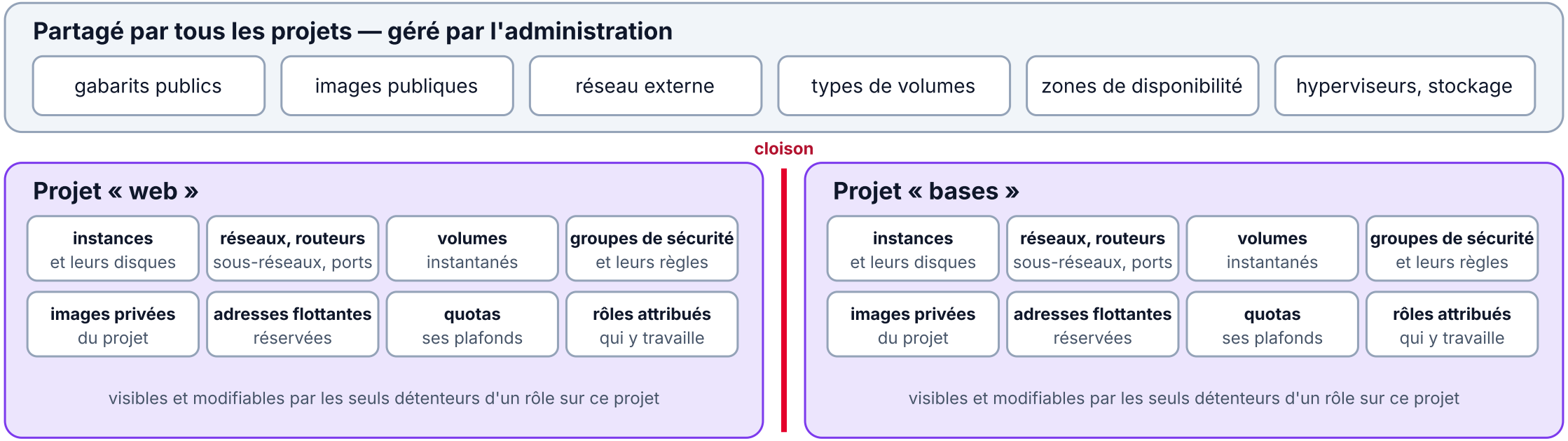
Ce qu'un projet porte, service par service

Service	Ressources appartenant au projet	Visible des autres projets ?	Plafonné par quota ?
Nova	**instances, groupes de serveurs ; paires de clés (par utilisateur)**	non	oui : instances, cœurs, mémoire
Cinder	**volumes, instantanés, sauvegardes**	non, sauf transfert accepté	oui : nombre et gigaoctets
Neutron	**réseaux, sous-réseaux, ports, routeurs, adresses flottantes, groupes de sécurité**	non, sauf partage explicite	oui : chaque type d'objet
Glance	**images privées**	selon la visibilité : privée, partagée, communautaire, publique	oui, par limites unifiées
Keystone	**rien : il décrit le projet lui-même**	visible dans son domaine	c'est lui qui stocke les limites unifiées
Octavia	**répartiteurs de charge, écouteurs, groupes de membres**	non	oui
Heat	**piles d'orchestration**	non	nombre de piles, par configuration
Barbican, Manila, Magnum, Designate	**secrets, partages de fichiers, clusters, zones DNS**	non	oui, propres à chaque service

Ce qui n'appartient à aucun projet : gabarits, hyperviseurs, agrégats, types de volumes, réseaux fournisseurs. Ce sont des ressources du cloud, gérées par l'administration.

Chaque service tient ses propres comptes : il n'existe pas de « quota global du projet ». Un projet a un jeu de quotas dans Nova, un autre dans Cinder, un autre dans Neutron.

Ce qui est étanche, ce qui est partagé



- **Réseau :** deux réseaux de deux projets ne communiquent pas, sauf par un routeur et une adresse externe, comme deux réseaux d'entreprises distinctes.
- **API :** lister les instances ne renvoie que celles du projet du jeton ; demander celle d'un autre renvoie « introuvable ».
- **Matériel :** la cloison est logique. Les instances de deux projets partagent hyperviseurs et disques ; l'étanchéité repose sur la virtualisation.
- **Entre membres d'un même projet :** aucune cloison. Tout membre voit et peut modifier toutes les ressources du projet.

Partager entre projets, volontairement

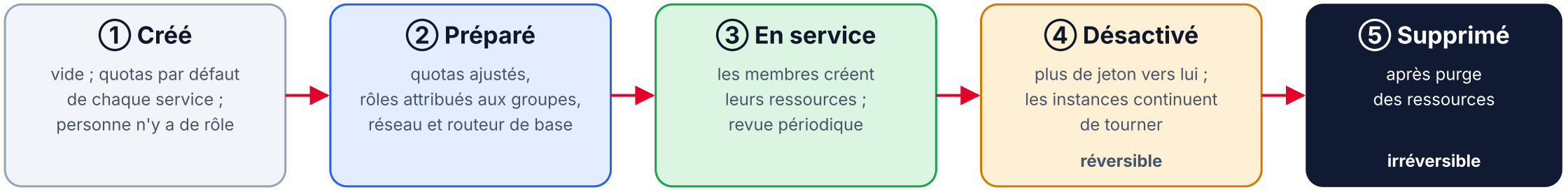
Ressource	Mécanisme	Commande type	Qui décide
Image	**visibilité « partagée » : le propriétaire ajoute un projet membre, qui accepte**	`openstack image add project`	le propriétaire, puis le destinataire
Réseau	**règle de partage ciblée vers un projet**	`openstack network rbac create`	le propriétaire du réseau
Groupe de sécurité, pool d'adresses	**même mécanisme de partage ciblé**	`openstack network rbac create`	le propriétaire
Volume	**transfert de propriété : un code est remis au projet destinataire**	`openstack volume transfer request create`	le propriétaire, puis le destinataire
Gabarit	**gabarit privé, ouvert à une liste de projets**	`openstack flavor set --project`	l'administration
Type de volume	**type privé, ouvert à une liste de projets**	`openstack volume type set --project`	l'administration
Accès humain	**un rôle dans les deux projets**	`openstack role add`	gestionnaire du domaine

```
# Ouvrir le réseau « commun » du projet socle au projet web
openstack network rbac create --type network \
  --action access_as_shared \
  --target-project web --target-project-domain nord \
  commun

# Voir ce qui est partagé, et avec qui
openstack network rbac list --long
```

- **Le partage est une exception explicite :** il se déclare ressource par ressource et projet par projet, et se retire de même.
- **Le quota reste chez le propriétaire :** une ressource partagée compte dans le projet qui la possède ; les ports créés dessus comptent chez celui qui les crée.

Cycle de vie d'un projet



Supprimer un projet dans Keystone ne supprime **pas** ses ressources dans les autres services : elles deviennent orphelines.

```
# Prérequis : jeton d'administrateur ou de gestionnaire de domaine
openstack project create --domain nord \
  --description "Site vitrine" vitrine
openstack project show vitrine --domain nord

# Suspendre, reprendre
openstack project set --disable vitrine --domain nord
openstack project set --enable  vitrine --domain nord

# Retirer : purger d'abord, supprimer ensuite
openstack project purge --dry-run --project <id>
openstack project purge --project <id>
openstack project delete <id>
```

- **Désactiver n'arrête rien :** c'est l'accès par l'API qui est coupé. Pour libérer de la capacité, il faut arrêter ou supprimer les instances.
- `project purge` supprime instances, volumes, instantanés et images du projet ; l'option `--dry-run` liste sans agir. Réseaux, routeurs, adresses flottantes, répartiteurs et secrets sont à vérifier à part.
- **Ordre de nettoyage réseau :** ports et adresses flottantes, interfaces de routeur, routeurs, sous-réseaux, réseaux.

> Un projet supprimé sans purge laisse des instances qui tournent et consomment, mais que plus personne ne peut atteindre sans droits d'administration.

Étiquettes et propriétés

Attribut	Rôle
identifiant	attribué par Keystone, immuable ; référence de toutes les ressources
nom	unique dans le domaine ; modifiable
domaine	fixé à la création, définitif
parent	le domaine ou un autre projet ; fixé à la création
description	texte libre
activé	interrupteur d'accès
étiquettes	mots-clés libres, filtrables
options	par exemple « immuable » : interdit modification et suppression

> **Les étiquettes compensent ce que la hiérarchie ne dit pas :** environnement, centre de coût, niveau de sensibilité, application. Elles servent ensuite à filtrer et à produire des rapports.

```
# Poser des étiquettes
openstack project set vitrine --domain nord \
  --tag env:prod --tag cout:cc-4410 --tag sens:interne

# Filtrer : tous les projets de production du domaine
openstack project list --domain nord --tags env:prod

# Ceux qui portent l'une OU l'autre étiquette
openstack project list --tags-any env:prod,env:rec

# Ceux qui n'en portent aucune : à corriger
openstack project list --domain nord --not-tags-any \
  env:prod,env:rec,env:dev

# Protéger un projet critique contre l'erreur de manipulation
openstack project set socle --domain nord --immutable
```

- **Convention à fixer une fois :** des préfixes — `env:`, `cout:`, `sens:`, `app:` — évitent les variantes d'écriture.
- **Une étiquette n'a aucun effet technique :** elle ne donne ni droit ni quota. C'est un classement.
- **Les services peuvent lire les étiquettes** par l'API de Keystone ; un outil de facturation s'en sert pour ventiler les coûts.

1 · LE PROJET Comptes : module 8 · domaines : module 9

Les projets d'un cloud Sunbeam

Projet	Créé par	Contenu	Règle de conduite
d'administration	**l'installation**	portée par défaut du compte `admin` ; ressources de l'administration : images publiques, réseau externe	ne pas y faire tourner de charge applicative
des services	**les charms**	portée des comptes de service ; ressources internes de certaines fonctionnalités	ne pas y toucher
de démonstration	`sunbeam configure`	un réseau privé, un routeur vers l'externe, des règles de sécurité ouvertes pour SSH et ping	sert à valider l'installation ; à retirer en production
les vôtres	**administrateur ou gestionnaires de domaine**	ce que les équipes y créent	dans vos domaines, jamais dans les domaines techniques

```
# Relever l'existant — prérequis : source admin-openrc
openstack project list --long
openstack quota show --default
openstack network list --external
openstack flavor list
openstack image list --public

# Ce que le projet de démonstration a reçu
source demo-openrc
openstack network list
openstack router list
openstack quota show
```

- `sunbeam configure` montre, sur un cas réel, ce qu'il faut préparer pour qu'un projet soit utilisable : un réseau, un sous-réseau, un routeur, des règles de sécurité.
- **Un projet neuf est vide :** ni réseau ni routeur. Tant qu'on ne lui en donne pas, ses membres ne peuvent lancer d'instance que sur un réseau partagé.
- **Noms exacts** des projets techniques : à lire avec la première commande, plutôt qu'à supposer.

> Les fonctionnalités optionnelles — équilibrage de charge, par exemple — créent leurs propres ressources dans le projet des services : c'est normal, et à ne pas « nettoyer ».

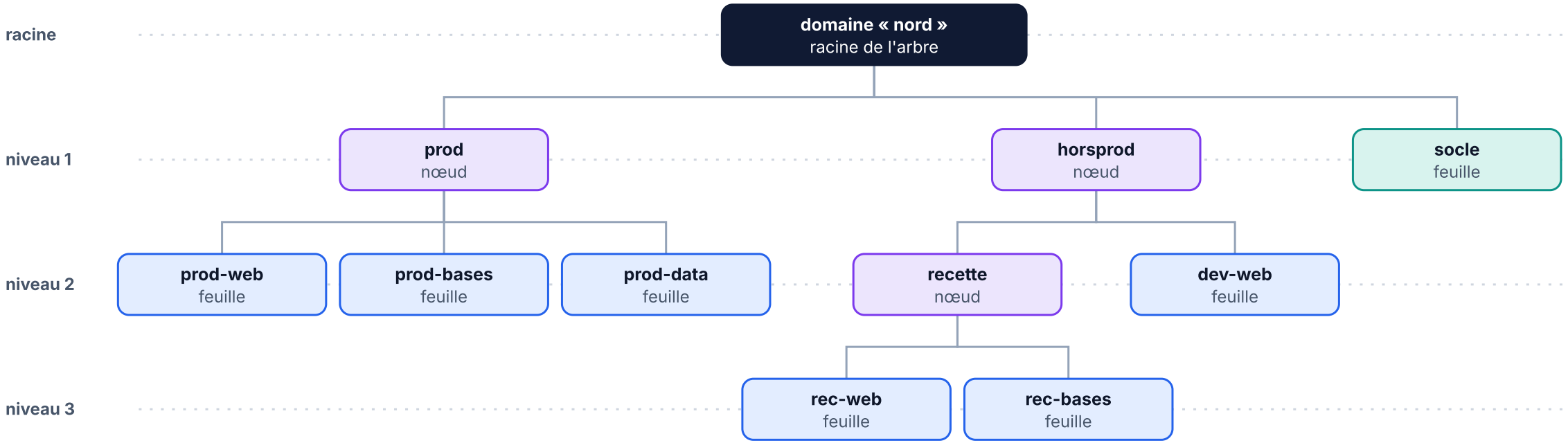
02

La hiérarchie de projets

Un projet peut en contenir d'autres. L'arbre ne change rien à l'isolation des ressources ; il sert à organiser, et surtout à faire descendre des droits.

- L'arbre de projets
- Les rôles hérités
- Ce que la hiérarchie ne fait pas
- Quand s'en servir
- Les règles de l'arbre
- Direct ou hérité : comparaison
- Consulter et manipuler l'arbre

L'arbre de projets



Lecture : le domaine est la racine ; chaque projet a exactement un parent ; les projets sans enfant — les feuilles — sont en général ceux qui portent les ressources.

Rien n'oblige à mettre des ressources dans un nœud intermédiaire : « production » peut n'être qu'un dossier, qui sert à ranger et à porter des rôles hérités.

Les règles de l'arbre

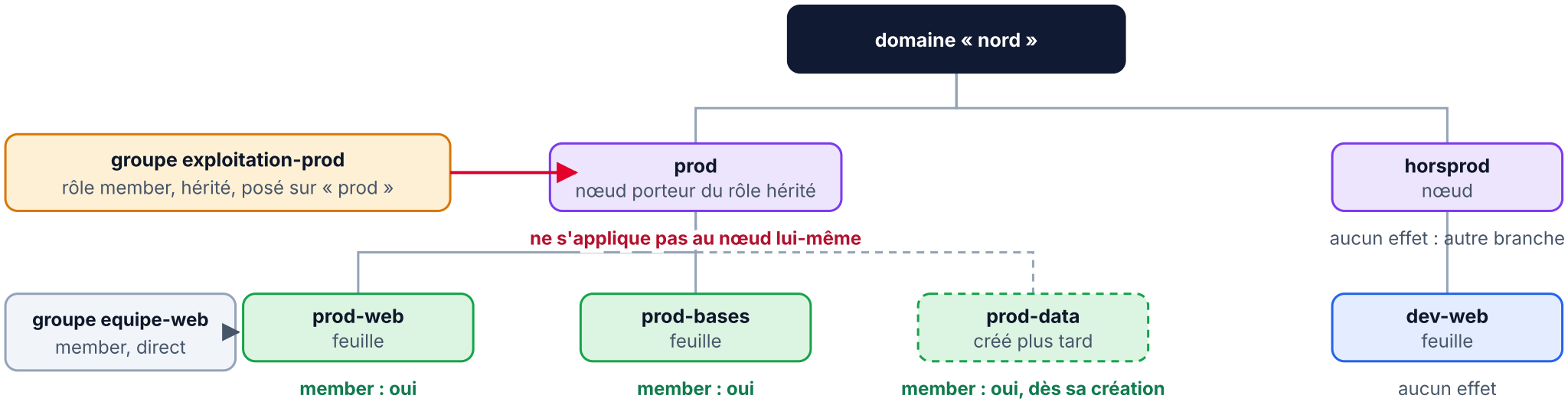
Règle	Détail	Conséquence pratique
Un seul parent	**fixé à la création du projet**	c'est un arbre, pas un graphe
Parent immuable	**on ne déplace pas un projet sous un autre parent**	concevoir l'arbre avant de créer ; sinon recréer et migrer
Même domaine	**un sous-projet est dans le domaine de son parent**	un arbre ne traverse pas les domaines
Noms uniques dans le domaine	**deux projets d'un même domaine ne peuvent porter le même nom, même sous des parents différents**	« prod/web » et « recette/web » : impossible ; préfixer ou suffixer
Profondeur limitée	**option `max_project_tree_depth` de Keystone, cinq niveaux par défaut**	deux ou trois niveaux suffisent presque toujours
Désactivation par les feuilles	**un projet désactivé ne peut pas avoir d'enfant actif**	pour suspendre une branche : désactiver les enfants, puis le parent
Suppression par les feuilles	**un projet qui a des enfants ne se supprime pas tel quel**	remonter l'arbre : feuilles d'abord
Isolation inchangée	**parent et enfant sont deux projets distincts pour tous les services**	le parent ne « voit » pas les instances de l'enfant

La règle qui surprend le plus : l'unicité des noms. L'arbre n'est pas un système de fichiers ; le nom complet d'un projet reste « nom + domaine », pas un chemin.

Règles décrites d'après le comportement connu de Keystone ; la profondeur maximale se vérifie sur votre version.

 Héritage depuis le domaine : module 9

Les rôles hérités



- **Un rôle hérité se pose sur un nœud et s'applique en dessous :** à tous ses descendants, présents et futurs — mais **pas** au nœud lui-même.
- **Pour avoir aussi le rôle sur le nœud,** on ajoute une affectation directe : les deux se cumulent.
- **Posé sur le domaine,** un rôle hérité couvre tous les projets du domaine.
- **Un nouveau sous-projet reçoit les droits sans rien faire :** c'est tout l'intérêt, et aussi le risque — il faut savoir ce qui descend avant de créer.

Direct ou hérité : commandes et lecture

```
# Hérité : les exploitants lisent toute la production
openstack role add --group exploitants --group-domain nord \
  --project production --project-domain nord \
  --inherited reader

# Direct : l'équipe web agit dans son projet seulement
openstack role add --group web-dev --group-domain nord \
  --project prod-web --project-domain nord member

# Hérité sur le domaine : l'audit lit tous les projets
openstack role add --group audit --group-domain nord \
  --domain nord --inherited reader
```

```
# Ce qui a été saisi sur un nœud
openstack role assignment list --names \
  --project production --project-domain nord
openstack role assignment list --names \
  --project production --project-domain nord --inherited

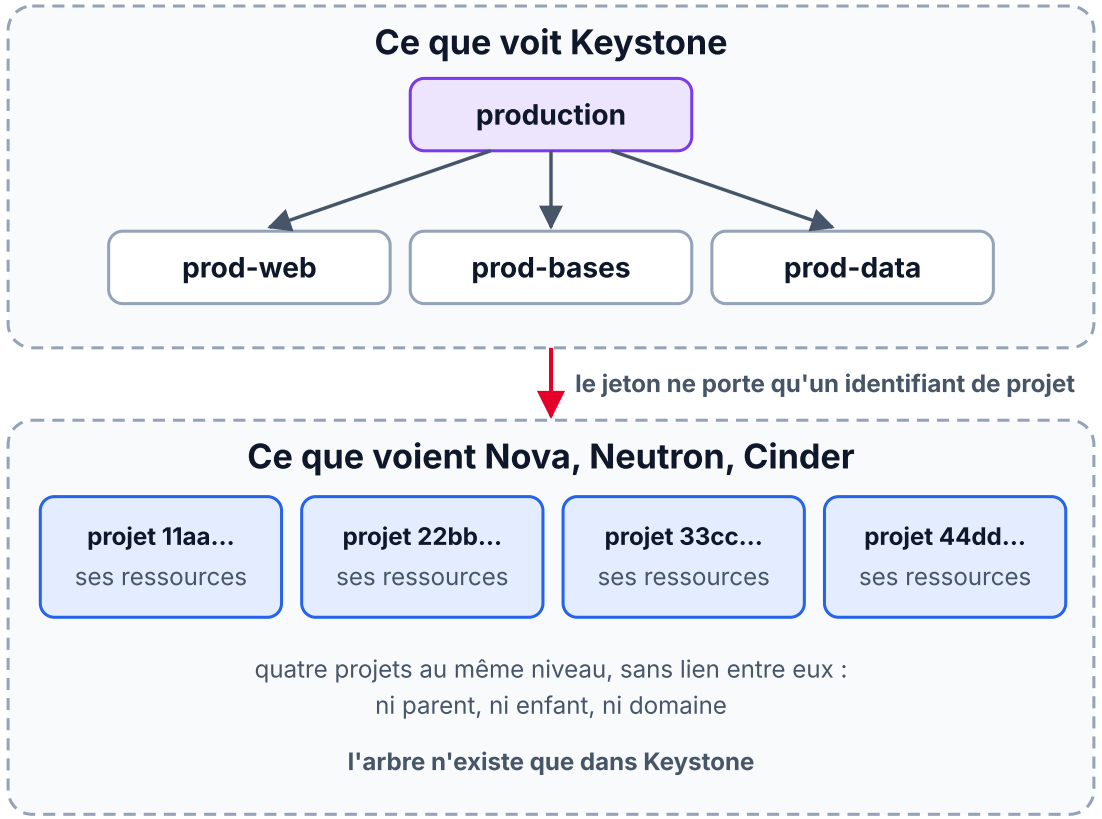
# Ce qui s'applique réellement sur une feuille
openstack role assignment list --names --effective \
  --project prod-web --project-domain nord
```

	Affectation directe	Affectation héritée
S'applique à	**la cible elle-même**	les descendants de la cible
Nouveaux sous-projets	**rien : à refaire**	couverts automatiquement
Lisibilité	**évidente**	exige `--effective` pour voir le résultat
Retrait d'un droit sur un seul enfant	**simple**	impossible : l'héritage ne se « coupe » pas
Usage type	**équipes applicatives**	fonctions transverses : exploitation, audit, sécurité

Pas d'exception possible : on ne peut pas dire « hérité partout sauf ici ». Si une feuille doit échapper à un droit, elle n'est pas sous la bonne branche.

Règle simple : lecture en hérité, large ; écriture en direct, ciblée.

Ce que la hiérarchie ne fait pas



On pourrait croire que…	En réalité
un membre du parent voit les instances des enfants	non : il lui faut un rôle hérité, ou direct, sur chaque enfant
le quota du parent borne les enfants	non avec les quotas classiques ; oui seulement avec les limites unifiées en mode strict
les réseaux du parent sont accessibles aux enfants	non : il faut un partage explicite
supprimer le parent supprime la branche	non : feuilles d'abord, et les ressources à part
la facture se cumule vers le haut	à faire par l'outil de facturation, en lisant l'arbre

> **À retenir :** la hiérarchie est un outil d'**organisation des droits**. Pour tout le reste — réseau, capacité, facturation — elle n'est qu'une information que d'autres mécanismes peuvent exploiter.

Consulter et manipuler l'arbre

```
# Prérequis : jeton d'administrateur ou de gestionnaire du domaine

# Construire une branche
openstack project create --domain nord production
openstack project create --domain nord \
  --parent production prod-web
openstack project create --domain nord \
  --parent production prod-bases

# Voir la position d'un projet
openstack project show prod-web --domain nord \
  -c id -c parent_id -c domain_id

# Ses ancêtres, sa descendance
openstack project show prod-web --domain nord --parents
openstack project show production --domain nord --children

# Les enfants directs d'un nœud
openstack project list --parent production --domain nord
```

```
# Tout l'arbre d'un domaine, sous forme lisible
openstack project list --domain nord --long \
  -c ID -c Name -c "Parent ID" -c Enabled

# Retirer une branche : feuilles d'abord
openstack project purge --project <id de prod-web>
openstack project delete <id de prod-web>
openstack project purge --project <id de prod-bases>
openstack project delete <id de prod-bases>
openstack project delete <id de production>
```

- **L'option `--parent` ne s'utilise qu'à la création :** il n'existe pas de commande pour déplacer un projet.
- **Colonnes affichées :** elles varient selon la version du client ; `-f json` donne toujours `parent_id`.
- **Pour un vrai dessin de l'arbre,** exporter en JSON et le reconstruire avec un petit script : c'est un bon exercice de travaux pratiques.

> Dans les commandes de suppression, préférer les identifiants : un nom se trompe plus facilement de domaine qu'un identifiant.

Quand se servir de la hiérarchie

Elle apporte quelque chose quand…

- **Des fonctions transverses** — exploitation, sécurité, audit — doivent avoir le même droit sur un ensemble de projets qui grandit.
- **Des projets naissent souvent :** un rôle hérité évite d'oublier une affectation à chaque création.
- **Une capacité doit être partagée** entre projets d'une même équipe, avec les limites unifiées en mode à deux niveaux.
- **Une branche doit pouvoir être suspendue ensemble** — un environnement, une application en fin de vie : l'arbre donne la liste, on désactive les feuilles puis le nœud.
- **Les rapports se font par regroupement :** coût par application, par environnement.

Elle complique sans rien apporter quand…

- **Il y a peu de projets,** stables : des affectations directes se lisent mieux.
- **Chaque projet a sa propre équipe,** sans fonction transverse.
- **L'arbre voudrait refléter l'organigramme :** il change plus vite que les projets, et un projet ne se déplace pas.
- **On attend d'elle une isolation** qu'elle ne fournit pas.
- **Les étiquettes suffisent** au classement dont on a besoin.

Besoin	Hiérarchie	Étiquettes	Domaines
Faire descendre un droit	**oui : rôles hérités**	non	oui, sur tout le domaine
Classer selon plusieurs axes	**un seul axe : l'arbre**	oui, autant d'axes que voulu	non
Changer de classement plus tard	**non : parent immuable**	oui, librement	non
Déléguer l'administration	**non**	non	oui
Partager un plafond	**oui, limites unifiées à deux niveaux**	non	oui, limite de domaine

Combinaison fréquente : un arbre peu profond pour l'axe qui porte les droits, des étiquettes pour tous les autres axes.

03

Les quotas

Un plafond par projet et par type de ressource. Sans eux, le premier projet venu peut consommer tout le cloud ; mal réglés, ils bloquent les équipes ou ne protègent rien.

- À quoi servent les quotas
- Calcul, stockage, réseau
- Lire, fixer, modifier
- Les limites unifiées
- Dimensionner
- Deux générations de mécanismes
- Comment un quota est vérifié
- Les quotas par défaut
- Quota et capacité réelle
- Pannes typiques

À quoi servent les quotas

Protéger la capacité

Aucun projet ne peut, par erreur ou par excès, épuiser les hyperviseurs, le stockage ou les adresses.

Répartir équitablement

La capacité achetée ou allouée à chaque équipe est matérialisée et opposable.

Borner les erreurs

Une boucle dans un script d'automatisation s'arrête au quota, pas à la saturation du cloud.

Maîtriser les coûts

Le plafond de consommation est aussi un plafond de dépense, connu d'avance.

Un quota est un plafond, pas une réservation



La somme des quotas — 600 — dépasse la capacité — 400. C'est permis, et courant : tous les projets ne consomment pas leur maximum en même temps.
Mais rien ne garantit alors à un projet qu'il pourra atteindre son quota.

Deux réglages distincts : le quota dit ce qu'un projet a **le droit** de demander ; la capacité dit ce que le cloud **peut** fournir. Les deux refus n'ont pas le même message d'erreur.

Chiffres d'illustration. Le rapport entre somme des quotas et capacité est un choix d'exploitation, à suivre dans le temps.

Deux générations de mécanismes

Quotas « classiques » — par service

Nova
ses limites, dans sa base
instances, cœurs…

Cinder
ses limites, dans sa base
volumes, Go…

Neutron
ses limites, dans sa base
réseaux, ports…

chaque service stocke, vérifie et expose ses quotas ;
trois API, trois jeux de valeurs par défaut

le plus répandu ; projets tous au même niveau

commande : openstack quota …

Limites unifiées — centralisées dans Keystone

Keystone stocke toutes les limites
par défaut, par domaine, par projet

Nova
compte et vérifie

Glance
compte et vérifie

autres services
à mesure qu'ils l'adoptent

un seul endroit pour définir ; chaque service lit la limite
et connaît la hiérarchie des projets

commandes : openstack registered limit … , openstack limit …

	Quotas classiques	Limites unifiées
Où sont les valeurs	**dans chaque service**	dans Keystone
Hiérarchie de projets	**ignorée**	exploitable, selon le modèle choisi
Plafond par domaine	**non**	oui
Adoption	**tous les services**	Glance ; Nova sur option ; les autres progressivement

> **Sur un cloud donné, les deux coexistent :** il faut savoir, service par service, lequel est actif. Les slides suivantes décrivent d'abord les quotas classiques, puis les limites unifiées.

Quotas de calcul — Nova

Quota	Ce qu'il plafonne	Défaut amont
`instances`	**nombre d'instances du projet**	10
`cores`	**somme des cœurs virtuels des instances**	20
`ram`	**somme de la mémoire, en mégaoctets**	51 200
`key-pairs`	**paires de clés — par utilisateur**	100
`server-groups`	**groupes de serveurs (affinité)**	10
`server-group-members`	**instances par groupe**	10
`properties`	**métadonnées par instance**	128

- **Les trois premiers font tout le travail :** une instance est refusée dès que l'un d'eux serait dépassé.
- **Le gabarit décide de la consommation :** une instance « 4 cœurs, 8 Gio » retire 1, 4 et 8192 des trois compteurs.
- **Instance arrêtée :** elle compte toujours. Seule une instance « remisée et déchargée » ne consomme plus cœurs ni mémoire.

```
# Quotas de calcul d'un projet
openstack quota show --compute prod-web

# Fixer
openstack quota set prod-web \
  --instances 40 --cores 160 --ram 327680
```

Cohérence à vérifier : 40 instances, 160 cœurs, 320 Gio correspondent à un gabarit moyen de 4 cœurs et 8 Gio. Des quotas incohérents entre eux rendent l'un des trois inutile.

Ce qui n'est pas plafonné ici : le disque éphémère des instances — sauf avec les limites unifiées — et les cartes graphiques ou autres ressources spéciales, qui se bornent par gabarits privés.

Valeurs par défaut du code amont, données de mémoire : celles de votre cloud se lisent avec `openstack quota show --default .`

Quotas de stockage — Cinder et Glance

Cinder : volumes

Quota	Ce qu'il plafonne	Défaut amont
`volumes`	nombre de volumes	10
`gigabytes`	taille cumulée des volumes et des instantanés	1000
`snapshots`	nombre d'instantanés	10
`backups`	nombre de sauvegardes	10
`backup-gigabytes`	taille cumulée des sauvegardes	1000
`per-volume-gigabytes`	taille maximale d'un volume	illimitée

- **Par type de volume :** chaque type a en plus ses propres compteurs — utile pour rationner un stockage rapide et coûteux.
- **Les instantanés comptent dans `gigabytes` :** une équipe qui en abuse se bloque elle-même.

```
openstack quota show --volume prod-web
openstack quota set prod-web \
  --volumes 80 --gigabytes 4000 --snapshots 80

# Quota propre à un type de volume
openstack quota set prod-web \
  --volume-type rapide --gigabytes 500
```

Glance : images

Limite	Ce qu'elle plafonne
`image_size_total`	espace total des images du projet, en Mio
`image_stage_total`	espace des téléversements en cours
`image_count_total`	nombre d'images
`image_count_uploading`	téléversements simultanés

> Glance ne propose ses quotas par projet **que** par les limites unifiées : ils se règlent dans Keystone, pas avec `openstack quota set` .

> Sur un stockage Ceph, les volumes sont alloués à la demande : le quota borne la taille **promise**, pas l'espace réellement écrit.

Quotas réseau — Neutron

Quota	Ce qu'il plafonne	Défaut amont
`networks`	**réseaux du projet**	100
`subnets`	**sous-réseaux**	100
`ports`	**ports : un par carte d'instance, plus ceux des routeurs, du DHCP, des répartiteurs**	500
`routers`	**routeurs**	10
`floating-ips`	**adresses flottantes — donc adresses externes**	50
`secgroups`	**groupes de sécurité**	10
`secgroup-rules`	**règles, tous groupes confondus**	100
`subnetpools` , `rbac-policies`	**pools de sous-réseaux, règles de partage**	illimité, 10

- **Le quota qui compte vraiment : les adresses flottantes.** Elles puisent dans la plage du réseau externe, ressource rare et commune à tous.
- **Le quota qui bloque sans prévenir : les ports.** Une instance sans port disponible échoue à la création avec une erreur réseau, pas une erreur de calcul.

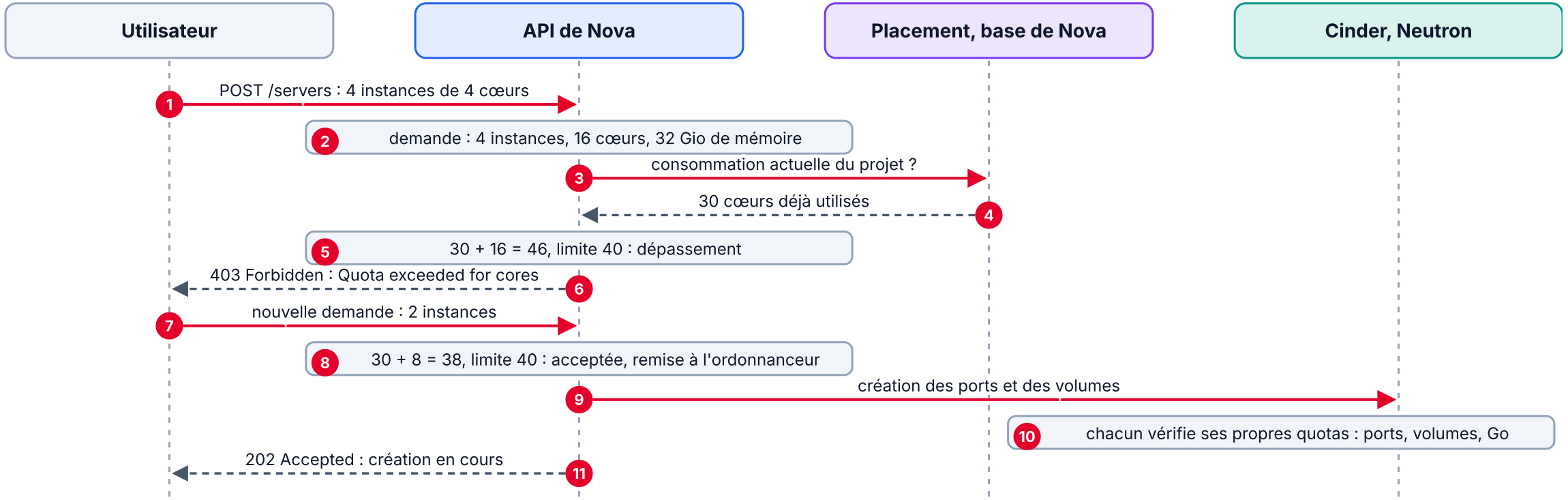
```
openstack quota show --network prod-web
openstack quota set prod-web \
  --networks 10 --subnets 20 --routers 3 \
  --ports 300 --floating-ips 8 \
  --secgroups 30 --secgroup-rules 300
```

Règle de cohérence : prévoir au moins un port par instance autorisée, plus une marge pour routeurs, DHCP et répartiteurs. Avec 40 instances, 300 ports sont confortables ; 40 seraient un piège.

Somme des adresses flottantes : la somme des quotas de tous les projets ne devrait pas dépasser de beaucoup la taille de la plage externe ; c'est le seul quota que l'on évite de surallouer.

L'équilibrage de charge a ses propres quotas, dans son propre service, lorsqu'il est activé.

Comment un quota est vérifié



- **La vérification se fait à l'entrée**, dans l'API du service, avant toute planification : un refus de quota ne laisse aucune trace d'instance.
- **Le service compte lui-même la consommation** à chaque demande — dans ses bases, ou auprès de Placement pour les cœurs et la mémoire.
- **Deux refus à distinguer :** « Quota exceeded », code 403, vient du quota ; « No valid host », instance en erreur, vient de la capacité.
- **Opération composée :** créer une instance avec un volume et une adresse flottante interroge trois quotas, dans trois services.

Lire les quotas et la consommation

```
# Prérequis : un rôle dans le projet, ou d'administration

# Les limites d'un projet, tous services
openstack quota show prod-web

# Limites ET consommation
openstack quota show --usage prod-web

# Les valeurs par défaut du cloud
openstack quota show --default

# Par famille
openstack quota show --compute --usage prod-web
openstack quota show --volume  --usage prod-web
openstack quota show --network --usage prod-web

# Vue « utilisateur » du projet courant
openstack limits show --absolute

# Projets dont les quotas diffèrent du défaut (administrateur)
openstack quota list --compute
openstack quota list --network
```

```
# Extrait type de « quota show --usage »
+----------------+--------+--------+----------+
| Resource       | In Use | Reserved | Limit  |
+----------------+--------+--------+----------+
| cores          |    112 |      0 |      160 |
| instances      |     28 |      0 |       40 |
| ram            | 229376 |      0 |   327680 |
| volumes        |     35 |      0 |       80 |
| gigabytes      |   2150 |      0 |     4000 |
| floating-ips   |      6 |      0 |        8 |
| ports          |     74 |      0 |      300 |
+----------------+--------+--------+----------+
```

Colonne	Signification
In Use	ce que possède le projet en ce moment
Reserved	ce qui est engagé par des opérations en cours
Limit	le plafond ; `-1` signifie illimité

Dans Horizon, la page « Vue d'ensemble » d'un projet affiche les mêmes chiffres sous forme de jauges : c'est ce que consultent les utilisateurs.

Sortie reconstituée pour l'exemple ; la mise en forme exacte dépend de la version du client.

Fixer et modifier un quota

```
# Prérequis : identifiants d'administrateur
source admin-openrc
PID=$(openstack project show prod-web --domain nord -f value -c id)

# Tout en une commande : calcul, volumes, réseau
openstack quota set $PID \
  --instances 40 --cores 160 --ram 327680 \
  --volumes 80 --gigabytes 4000 --snapshots 80 \
  --networks 10 --subnets 20 --routers 3 --ports 300 \
  --floating-ips 8 --secgroups 30 --secgroup-rules 300

# Vérifier
openstack quota show --usage $PID

# Illimité — à éviter
openstack quota set $PID --instances -1

# Revenir aux valeurs par défaut
openstack quota delete $PID
```

Situation	Comportement
Quota relevé	**effet immédiat pour les nouvelles demandes**
Quota abaissé sous la consommation	**rien n'est supprimé ; toute nouvelle création est refusée jusqu'à repasser sous le plafond**
Quota à zéro	**interdit ce type de ressource au projet**
Quota jamais fixé	**le projet suit le défaut — et suivra ses changements**
Quota fixé explicitement	**il ne suit plus le défaut**

- **Qui peut le faire :** l'administration du cloud. Un gestionnaire de domaine ou un membre de projet peut lire ses quotas, pas les changer.
- **Toujours par identifiant** dans les scripts : le nom d'un projet n'est unique que dans son domaine.
- **Abaisser pour interdire :** mettre `floating-ips` à 0 est la manière la plus simple d'empêcher un projet de s'exposer à l'extérieur.

> Tracer chaque changement : qui l'a demandé, pourquoi, pour combien de temps. Un quota relevé « provisoirement » ne redescend jamais tout seul.

Les quotas par défaut : le réglage le plus important



- **Créer un projet est une opération déléguée ; fixer un quota ne l'est pas.** Entre les deux, le projet vit avec le défaut.
- **Un défaut généreux multiplie la capacité promise** par le nombre de projets, sans aucun contrôle.
- **Défaut des adresses flottantes :** cinquante par projet épuisent une petite plage externe en un projet.

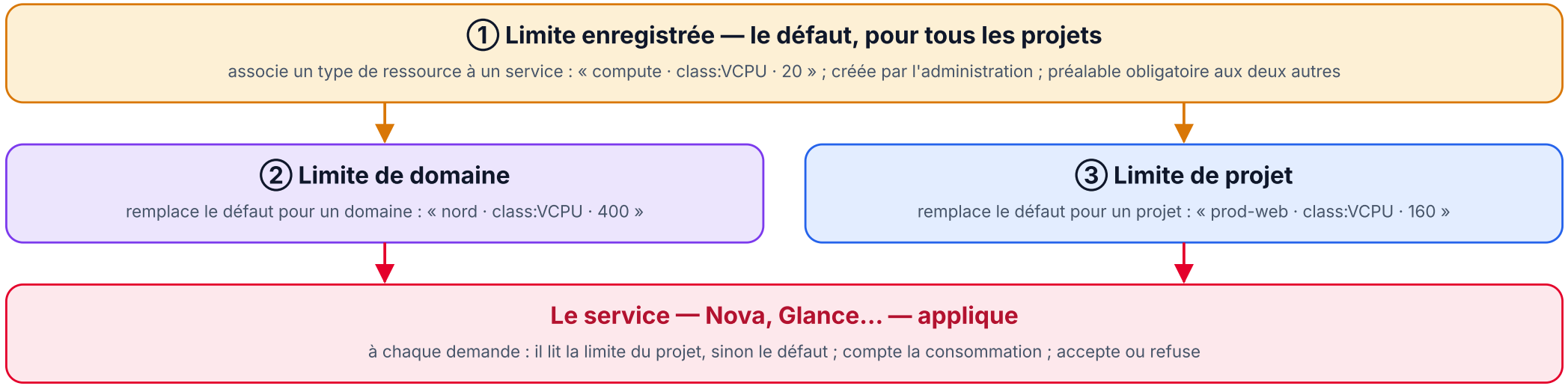
Stratégie	Défaut	Effet
Ouverte	**valeurs amont**	tout nouveau projet est utilisable ; capacité non maîtrisée
Bac à sable	**très bas : 2 instances, 4 cœurs, 0 adresse flottante**	on peut essayer ; tout usage réel passe par une demande
Fermée	**zéro partout**	rien ne fonctionne sans quota explicite ; contrôle total

```
# Lire les défauts en vigueur
openstack quota show --default

# Les changer : défauts de calcul et de volumes
openstack quota set --class default \
  --instances 2 --cores 4 --ram 8192 \
  --volumes 2 --gigabytes 40
```

Sur Sunbeam : certains défauts — ceux de Neutron notamment — sont des options de configuration du service, donc du charm. À vérifier avant de compter sur la commande ci-dessus.

Les limites unifiées : trois niveaux de valeurs



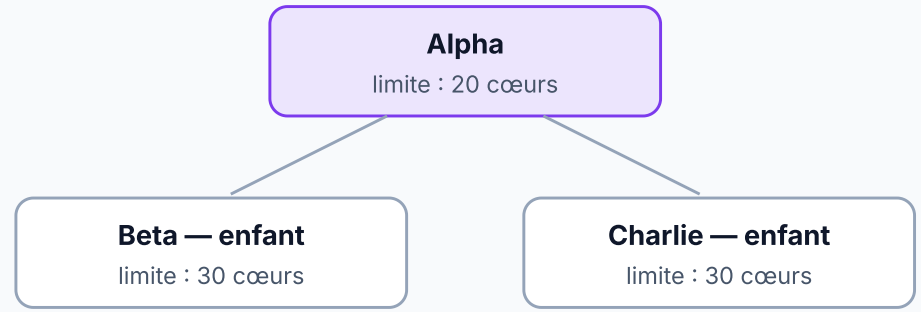
- **Keystone stocke et valide les limites ; il ne compte rien.** La consommation reste l'affaire de chaque service.
- **Valeurs :** de 0 à un très grand nombre ; `-1` pour illimité.
- **Abaisser sous la consommation** ne détruit rien : les nouvelles demandes sont refusées.

Ressource (Nova)	Équivalent classique
`servers`	instances
`class:VCPU`	cores
`class:MEMORY_MB`	ram
`class:DISK_GB`	— nouveau : disque des instances
`server_key_pairs`, `server_groups`…	key-pairs, server-groups…

> Toute classe de ressource de Placement peut être plafonnée ainsi, cartes graphiques comprises.

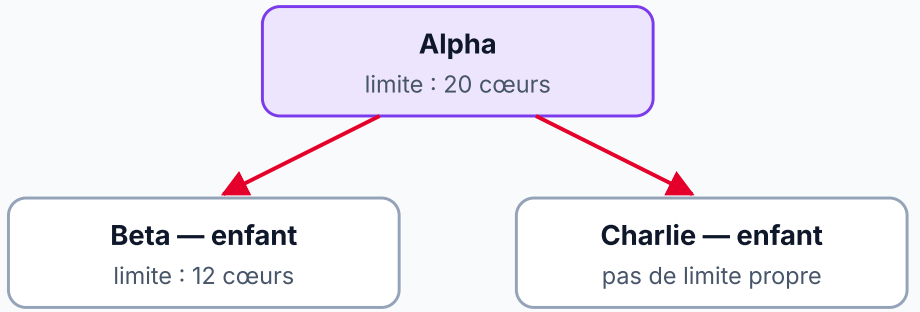
Limites unifiées : deux modèles d'application

Modèle « flat » — à plat



chaque projet est jugé seul : la limite du parent
ne borne pas les enfants, qui peuvent le dépasser
total possible : 20 + 30 + 30 = 80 cœurs

Modèle « strict_two_level » — deux niveaux stricts



la consommation de tout l'arbre est plafonnée par le parent ;
aucun enfant ne peut dépasser la limite du parent
total possible : 20 cœurs — Charlie dispose de ce qui reste

	flat	strict_two_level
Rapport parent / enfant	**aucun**	le parent borne la somme de l'arbre
Profondeur de l'arbre	**quelconque**	**deux niveaux au plus** : créer un petit-enfant est refusé
Partage de capacité	**manuel, limite par limite**	automatique à l'intérieur de l'arbre
Validation des valeurs	**aucune**	à la création et à la modification
Usage type	**projets indépendants**	une enveloppe par équipe ou par domaine, répartie librement

Un seul modèle pour tout le cloud, choisi dans la configuration de Keystone ; en changer après coup peut invalider des limites existantes. Le modèle strict contraint aussi la forme des arbres de projets : à décider avant de les dessiner.

Limites unifiées : commandes et points d'attention

```
# Prérequis : identifiants d'administrateur
source admin-openrc

# Quel modèle est en vigueur ? (ou keystone.conf, [unified_limit])
curl -s -H "X-Auth-Token: $(openstack token issue -f value -c id)" \
  $OS_AUTH_URL/limits/model
openstack limit list              # limites de projet existantes
openstack registered limit list   # défauts enregistrés

# 1 · Enregistrer les défauts — un par ressource
openstack registered limit create \
  --service nova --default-limit 2  servers
openstack registered limit create \
  --service nova --default-limit 4  class:VCPU
openstack registered limit create \
  --service nova --default-limit 8192 class:MEMORY_MB

# 2 · Relever la limite d'un projet
openstack limit create --project $PID \
  --service nova --resource-limit 160 class:VCPU

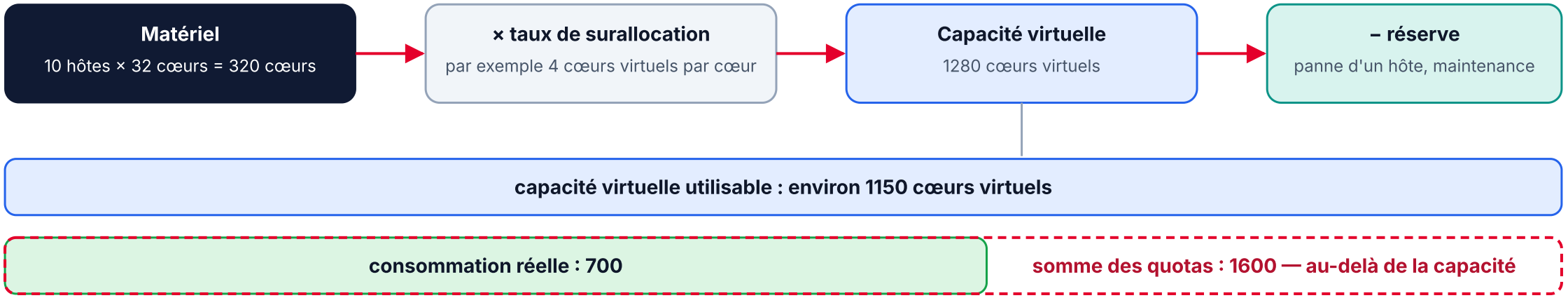
# 3 · Modifier, supprimer
openstack limit set --resource-limit 200 <id de la limite>
openstack limit delete <id de la limite>
```

Point d'attention	Détail
Activation dans Nova	**sur option : le pilote de quotas se choisit dans la configuration du service**
Ressource sans défaut enregistré	**pour les ressources déclarées obligatoires — `servers` par défaut — la limite vaut alors zéro : tout est refusé**
Bascule depuis les quotas classiques	**un outil de migration recopie les quotas de Nova dans Keystone**
Quotas par utilisateur	**n'existent plus dans ce modèle**
Comptage	**par Placement**

Sur Sunbeam : le mécanisme actif dans chaque service dépend de la configuration posée par les charms. Vérifier quel pilote est en place : section `[quota]` de `nova.conf`, dans le pod du service.

Les deux jeux de commandes ne se mélangent pas :
`openstack quota set` agit sur les quotas classiques,
`openstack limit …` sur les limites unifiées.

Quota et capacité réelle



chiffres d'illustration ; les taux de surallocation sont des options de Nova, posées par le déploiement

Ressource	Surallocation	Remarque
Cœurs	courante	les instances n'utilisent pas tous leurs cœurs en permanence
Mémoire	à éviter	une mémoire promise deux fois finit en instance tuée
Disque sur Ceph	courante	allocation à la demande : surveiller l'espace réellement écrit
Adresses externes	impossible	une adresse ne se partage pas

- **Deux surallocations se cumulent :** celle du matériel (cœurs virtuels par cœur physique) et celle des quotas (somme des quotas par rapport à la capacité).
- **Indicateurs à suivre :** consommation ÷ capacité, pour savoir quand acheter ; somme des quotas ÷ capacité, pour savoir ce que l'on a promis.
- **Quand la capacité manque avant le quota :** l'instance est créée puis passe en erreur, « aucun hôte valide ». C'est une alerte d'exploitation, pas une erreur de l'utilisateur.

Dimensionner les quotas d'un projet

Méthode : partir du besoin, pas des valeurs par défaut

Étape	Question
1 · Inventaire	combien d'instances, de quel gabarit, en régime normal ?
2 · Pointe	combien en plus lors d'un déploiement, d'un test de charge ?
3 · Stockage	combien de volumes, de quelle taille ; quelle politique d'instantanés ?
4 · Réseau	combien de réseaux, de points d'exposition externes ?
5 · Marge	20 à 30 % au-dessus de la pointe
6 · Cohérence	cœurs et mémoire correspondent-ils au nombre d'instances ? ports ≥ instances ?

> **Un déploiement par remplacement** — créer les nouvelles instances avant de supprimer les anciennes — double brièvement la consommation : c'est la pointe la plus souvent oubliée.

Exemple : une application web

Besoin	Calcul	Quota
12 instances de 4 cœurs, 8 Gio	**régime normal**	—
Déploiement par remplacement	**12 × 2 = 24**	—
Marge de 25 %	**24 × 1,25 = 30**	**instances : 30**
Cœurs	**30 × 4**	**cores : 120**
Mémoire	**30 × 8192**	**ram : 245 760**
Un volume de 50 Go par instance	**30 × 50, plus instantanés**	**gigabytes : 3000**
Ports	**30 instances, 2 répartiteurs, routeur, DHCP**	**ports : 100**
Exposition	**2 répartiteurs, 1 bastion**	**floating-ips : 4**

- **Quota trop juste :** déploiements bloqués, demandes de relèvement en urgence.
- **Quota trop large :** il ne protège plus rien. Un quota jamais approché à 50 % est à revoir à la baisse.

Pannes typiques liées aux quotas

Symptôme	Cause probable	Vérification
« Quota exceeded for cores »	**plafond de cœurs atteint, même s'il reste des instances**	`openstack quota show --compute --usage`
instance en erreur, « No valid host »	**ce n'est pas le quota : la capacité manque, ou aucun hôte ne convient au gabarit**	état des hyperviseurs ; agrégats et gabarit
instance en erreur à l'étape réseau	**quota de ports atteint**	`openstack quota show --network --usage`
impossible d'obtenir une adresse flottante	**quota du projet atteint, ou plage externe épuisée**	quota ; adresses libres du sous-réseau externe
création de volume refusée alors qu'il reste des volumes	**plafond en gigaoctets atteint, instantanés compris**	`openstack quota show --volume --usage`
quota modifié, sans effet	**mauvais projet — même nom, autre domaine — ou mauvais mécanisme : classique au lieu d'unifié**	identifiant du projet ; pilote de quotas actif
tout est refusé dans un projet neuf	**défaut à zéro, ou ressource sans limite enregistrée avec les limites unifiées**	`quota show --default` ; `registered limit list`
consommation affichée supérieure au réel	**instances arrêtées, supprimées mais non purgées, redimensionnement en cours**	`openstack server list --all` dans le projet
déploiement automatisé qui échoue à mi-parcours	**pointe de consommation pendant le remplacement des instances**	marge prévue pour un déploiement par remplacement

Premier réflexe : lire le message jusqu'au bout — il nomme la ressource en cause — puis comparer consommation et limite pour cette ressource précise, dans le bon service.

04

Organiser un système d'information cloisonné

Projets, hiérarchie et quotas sont des briques. Reste à décider comment les assembler pour que chaque équipe, chaque application, chaque environnement soit séparé des autres — et à compléter par le réseau et le matériel.

- Quatre niveaux de cloisonnement
- Quatre modèles d'organisation
- Cloisonner le réseau, le matériel
- Production et hors production
- Les axes de découpage
- Comparaison et modèle recommandé
- Services communs
- Nommage, étiquettes, scénario chiffré

Quatre niveaux de cloisonnement

	Ce qu'il sépare	Mécanisme OpenStack	Exemple de besoin
① **Identité** le domaine	annuaires, noms, administration des comptes	domaine Keystone, source d'identité par domaine	deux organisations, deux annuaires
② **Ressources** le projet	propriété, visibilité dans les API, plafonds	projet, rôles, quotas — appliqués par chaque service	deux applications, deux environnements
③ **Réseau** les flux	ce qui peut communiquer avec quoi	réseaux par projet, routeurs, groupes de sécurité, RBAC	base de données non joignable d'Internet
④ **Matériel** les hôtes	sur quelles machines tournent les charges	agrégats d'hôtes, zones, gabarits privés	exigence réglementaire, licences par hôte

Les niveaux se cumulent, ils ne se remplacent pas. Deux projets d'un même domaine sont isolés pour les ressources, mais partagent un annuaire ; deux projets isolés partagent les mêmes hyperviseurs tant qu'on ne dit rien de plus.

Le projet est le niveau obligatoire : toute ressource appartient à un projet. Les trois autres se décident selon l'exigence — séparation d'annuaires, de flux, ou de machines physiques.

Les axes de découpage

Axe	Question posée	Ce qu'il sépare bien	Limite s'il est seul
Organisation	**qui paie, qui décide, qui administre ?**	budgets, responsabilités, annuaires	une équipe mélange production et essais dans le même projet
Application	**quel système cette ressource fait-elle fonctionner ?**	cycle de vie, propriétaire technique, flux réseau	les environnements d'une même application restent ensemble
Environnement	**production, recette, développement ?**	droits d'accès, niveau d'exigence, quotas	toutes les applications de production partagent un seul plafond
Sensibilité	**données réglementées, exposition à Internet ?**	audit, matériel dédié, chiffrement	ne dit rien de qui exploite

Un projet = une réponse unique

Un bon projet a un seul propriétaire, un seul environnement, un seul niveau de sensibilité. Dès qu'une question appelle « ça dépend de la ressource », il faut deux projets.

Deux axes au moins

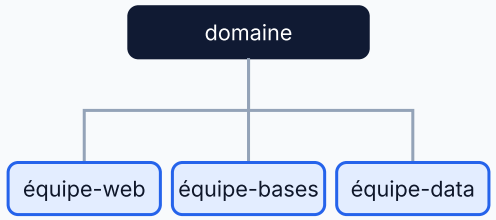
En pratique on croise **application** et **environnement** ; l'organisation se place au-dessus — domaine ou branche de l'arbre.

Le test décisif

« Si ce projet est compromis, ou si son quota est épuisé, qu'est-ce qui tombe avec lui ? » La réponse doit tenir en une ligne.

Quatre modèles d'organisation

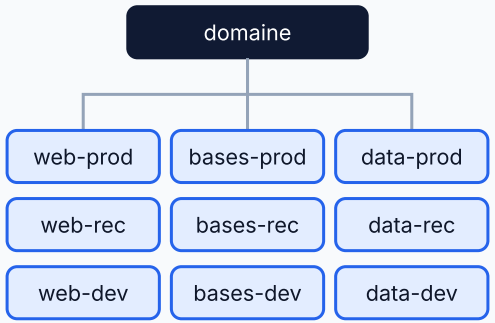
① Par équipe



production, recette et essais dans le même projet

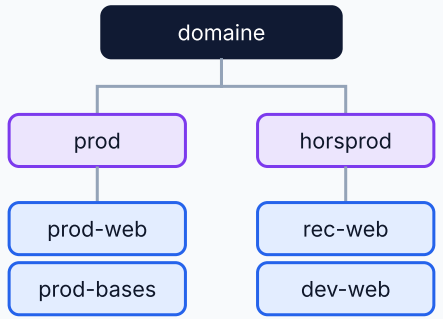
tout ce que fait l'équipe, au même endroit

② Application × environnement



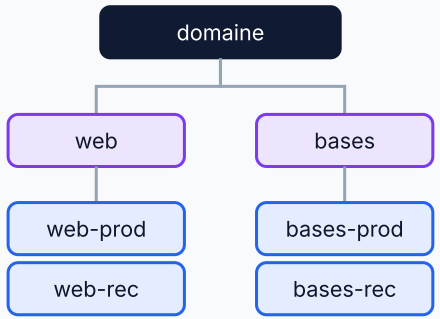
une case par couple, tout au même niveau

③ Arbre par environnement



environnement en nœud, application en feuille

④ Arbre par application



application en nœud, environnement en feuille

① Par équipe
Simple, vite en place. Convient à un cloud d'essai ; mélange les environnements.

② Application × environnement
Cloisonnement fin, à plat. Les droits se posent projet par projet.

③ Arbre par environnement
Un rôle hérité sur « production » couvre toute la production.

④ Arbre par application
Un rôle hérité sur « web » donne tous les environnements de l'application.

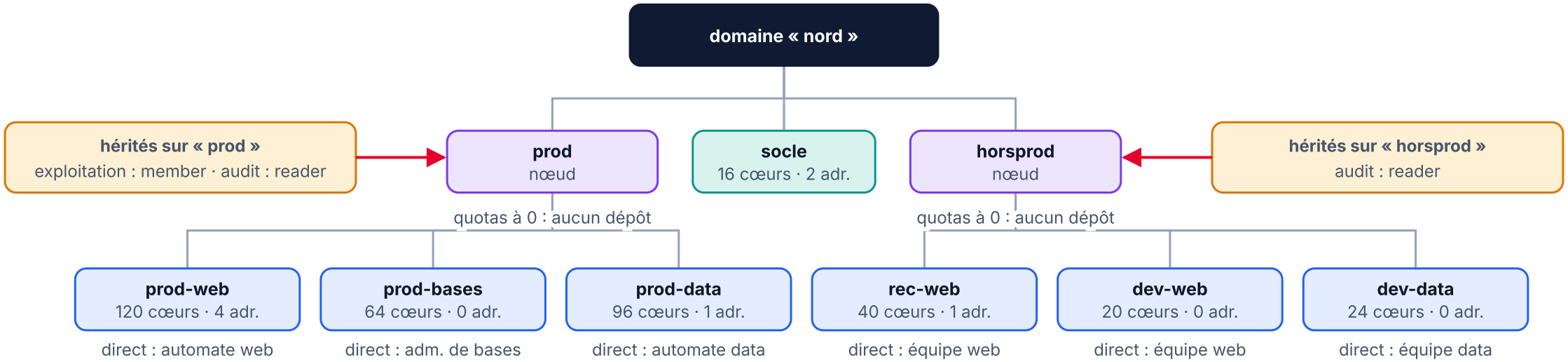
Comparer les quatre modèles

Critère	① Par équipe	② Application × environnement	③ Arbre par environnement	④ Arbre par application
Nombre de projets	**faible**	élevé	élevé, plus les nœuds	élevé, plus les nœuds
Production isolée des essais	**non**	**oui**	**oui**	**oui**
Incident limité à une application	**non**	**oui**	**oui**	**oui**
Affectations de rôles à gérer	**peu**	beaucoup : une par projet	peu : héritées par environnement	peu : héritées par application
Donner « toute la production » à l'exploitation	**impossible**	une affectation par projet	**une seule, héritée**	une par projet de production
Donner « toute l'application » à une équipe	**—**	une affectation par projet	une par projet de l'application	**une seule, héritée**
Plafond global par environnement	**non**	somme tenue à la main	**oui, avec** `strict_two_level`	non
Plafond global par application	**non**	somme tenue à la main	non	**oui, avec** `strict_two_level`
Risque principal	**un essai consomme le quota de la production**	dérive : droits oubliés, incohérents	un rôle hérité trop large en production	un développeur hérite de la production
Convient à	**laboratoire, équipe unique**	petit nombre d'applications	exploitation centralisée	équipes produit autonomes

Le choix entre ③ et ④ suit l'organisation : qui a le plus besoin d'un droit transversal — l'exploitation sur un environnement, ou une équipe sur son application ?

Le parent étant immuable, passer de ③ à ④ impose de recréer les projets. Le modèle se choisit avant la première création.

Un modèle de départ : l'arbre par environnement



- **Deux niveaux de projets, pas plus :** lisible, et compatible avec le modèle de limites `strict_two_level`.
- **Les nœuds ne portent aucune ressource :** quotas à zéro, ils rangent et transmettent.
- **Rôles hérités sur les nœuds** pour les populations transversales : exploitation, audit, sécurité.
- **Rôles directs sur les feuilles** pour les équipes applicatives.
- **Préfixe d'environnement dans le nom :** imposé par l'unicité des noms dans le domaine.
- **Une branche « socle »** pour ce qui sert à tous : images, réseau commun, supervision.

Cloisonner le réseau

Mécanisme	Rôle dans le cloisonnement
Réseau privé par projet	segment propre ; invisible des autres projets
Routeur du projet	seule sortie vers l'extérieur ; pas de routeur, pas de sortie
Groupes de sécurité	filtrage par instance ; tout ce qui entre est refusé par défaut
Adresses flottantes	seule manière d'être joint de l'extérieur ; quota à 0 = projet non exposable
Partage ciblé (RBAC)	ouvrir un réseau à un projet précis, pas à tous
Réseau externe	créé par l'administration ; peut lui aussi être réservé à certains projets

- **Deux projets ne se parlent pas par défaut :** il faut un routeur commun, un réseau partagé, ou passer par des adresses externes.
- **Le groupe de sécurité « default »** autorise les instances du même groupe à se parler : à revoir dans les projets sensibles.

```
# Projet non exposable : aucune adresse flottante
openstack quota set prod-bases --floating-ips 0

# Partager le réseau « commun » du socle
# avec un seul projet, pas avec tout le cloud
openstack network rbac create \
  --type network --action access_as_shared \
  --target-project prod-web  commun

# Réserver un réseau externe à un projet
openstack network rbac create \
  --type network --action access_as_external \
  --target-project prod-web  externe-prod

# Qui a accès à quoi ?
openstack network rbac list --long
```

> **Éviter `--share` à la création d'un réseau :** il l'ouvre à tous les projets, présents et futurs. Le partage ciblé dit explicitement qui entre.

> Sur Sunbeam, le réseau externe est créé par `sunbeam configure` ; les réseaux de projets sont des réseaux virtuels portés par OVN.

Cloisonner le matériel

Par défaut, tous les projets partagent tous les hyperviseurs. Trois outils, tous réservés à l'administration, permettent d'aller plus loin :

Outil	Effet
Agrégat d'hôtes	**groupe nommé d'hyperviseurs, porteur de propriétés ; invisible des utilisateurs**
Zone de disponibilité	**agrégat rendu visible : l'utilisateur la choisit à la création**
Agrégat réservé à des projets	**ces hôtes n'acceptent que les projets listés**
Gabarit privé	**gabarit visible des seuls projets autorisés ; peut cibler un agrégat par ses propriétés**
Type de volume privé	**même principe pour une classe de stockage**

Réserver un agrégat ne suffit pas : les hôtes refusent les autres projets, mais le projet réservataire peut encore être placé ailleurs. Pour une séparation dans les deux sens, lui donner uniquement des gabarits privés qui ciblent l'agrégat.

```
# Un agrégat pour la production sensible
openstack aggregate create prod-sensible
openstack aggregate add host prod-sensible noeud-4
openstack aggregate add host prod-sensible noeud-5

# Le réserver à un projet (identifiant)
openstack aggregate set prod-sensible \
  --property filter_tenant_id=$PID \
  --property classe=sensible

# Gabarit privé, qui cible l'agrégat
openstack flavor create s.4c8g --private \
  --vcpus 4 --ram 8192 --disk 40 \
  --property aggregate_instance_extra_specs:classe=sensible
openstack flavor set s.4c8g --project $PID
```

À vérifier sur votre déploiement : ces propriétés n'ont d'effet que si les filtres correspondants de l'ordonnanceur sont actifs. Sur Sunbeam, la configuration de Nova est écrite par les charms : contrôler par un essai avant de s'y fier.

Les services communs : le projet « socle »

Ce qui sert à tous ne doit appartenir à aucune application. Un projet — ou une branche — tenu par l'équipe plateforme porte les ressources communes.

Ressource commune	Mode de mise à disposition
Images de référence	visibilité « community » ou « shared », avec la liste des projets membres
Réseau d'administration	partage ciblé par RBAC, projet par projet
Bastion, supervision, journaux	instances du socle, jointes par le réseau commun
Gabarits, types de volumes	publics, ou privés et attribués
DNS, miroirs de paquets	instances du socle ; adresses publiées aux projets

Image « public » : réservée à l'administration du cloud. Une équipe plateforme sans rôle `admin` publie en « community » ou partage nommément.

```
# Partager une image du socle avec un projet
openstack image set --shared ubuntu-24.04-durcie
openstack image add project \
  ubuntu-24.04-durcie prod-web

# Côté projet destinataire : accepter
openstack image set --accept <id-image>
```

- **Le socle est un point de passage :** un réseau commun relie tous les projets qui y sont raccordés. Y filtrer strictement, sinon il annule le cloisonnement.
- **Un socle par niveau d'exigence :** la production et le hors-production ne partagent ni bastion ni réseau d'administration.
- **Droits sur le socle :** l'équipe plateforme seule ; les autres consomment, ils ne modifient pas.
- **Quotas du socle :** dimensionnés comme pour toute application — c'est un projet comme un autre.

Le socle remplace avantageusement le réflexe « tout mettre dans le projet admin », qui mêle outils communs et pouvoirs d'administration.

4 · ORGANISER UN SI CLOISONNÉ

Séparer production et hors production

Dimension	Production	Hors production	Moyen technique
Qui crée et supprime	**exploitation, automates de déploiement**	équipes de développement	groupes distincts, rôle `member`
Accès des développeurs	**lecture seule**	complet	rôle `reader` hérité sur la branche
Quotas	**calculés sur le besoin, marge comprise**	serrés, révisés souvent	quotas par projet ; plafond de branche
Exposition externe	**points d'entrée identifiés**	aucune, ou par bastion	quota d'adresses flottantes, réseau externe réservé
Réseau	**jamais partagé avec le hors-production**	réseaux propres	pas de routeur ni de RBAC commun
Matériel	**hôtes dédiés si l'exigence le demande**	hôtes mutualisés, surallocation plus forte	agrégats, gabarits privés
Identifiants d'automates	**propres à la production**	propres au hors-production	identifiants d'application par projet
Durée de vie des ressources	**longue, tracée**	courte, nettoyée régulièrement	étiquettes, revue périodique

Le critère de réussite : aucun compte, aucun réseau, aucun identifiant d'automate ne donne à la fois le droit d'écrire en production et hors production.

Exigence plus forte encore ? Deux domaines, voire deux clouds. Le projet sépare les ressources ; il ne sépare ni le plan de contrôle ni les administrateurs.

Conventions de nommage et étiquettes

Objet	Convention	Exemple
Nœud d'environnement	**nom de l'environnement**	`prod`, `horsprod`
Projet feuille	**environnement-application**	`prod-web`, `rec-web`
Groupe applicatif	**projet-niveau d'accès**	`prod-web-dev`, `prod-web-lecture`
Groupe transversal	**fonction-périmètre**	`exploitation-prod`
Réseau	**projet-usage**	`prod-web-front`
Agrégat	**environnement-classe**	`prod-sensible`

- **Le nom porte ce qui ne change pas :** environnement et application. Le reste — propriétaire, centre de coût, échéance — va dans les étiquettes.
- **Minuscules, tirets, pas d'espace :** les noms finissent dans des scripts.
- **Le nom d'un projet peut être modifié,** son parent non : une erreur de nom se corrige, une erreur de branche se paie.

```
# Étiqueter à la création
openstack project create prod-web \
  --domain nord --parent prod \
  --description "Site web — production" \
  --tag env:prod --tag app:web \
  --tag proprietaire:equipe-web --tag cout:cc-4410

# Retrouver tous les projets de production
openstack project list --tags env:prod

# Tous ceux d'une équipe, hors production
openstack project list \
  --tags proprietaire:equipe-web --not-tags env:prod
```

Étiquette	Sert à
`env:`	filtrer les revues, appliquer des gabarits de quotas
`app:`	regrouper les environnements d'une application
`proprietaire:`	savoir qui appeler ; revue des droits
`cout:`	refacturation, rapports de consommation
`echeance:`	repérer les projets temporaires à supprimer

Scénario chiffré : une organisation, trois applications

Projet	Parent	Instances	Cœurs	Mémoire	Volumes	Adr. flott.	Rôles directs	Rôles hérités reçus
`prod`	—	0	0	0	0	0	aucun	—
`prod-web`	`prod`	30	120	240 Gio	3 000 Go	4	automate de déploiement web	exploitation-prod : `member` audit : `reader` développeurs : `reader`
`prod-bases`	`prod`	8	64	256 Gio	6 000 Go	0	administrateurs de bases	
`prod-data`	`prod`	12	96	384 Gio	10 000 Go	1	automate de déploiement data	
`horsprod`	—	0	0	0	0	0	aucun	—
`rec-web`	`horsprod`	10	40	80 Gio	800 Go	1	équipe web	audit : `reader` exploitation-prod : `reader`
`dev-web`	`horsprod`	10	20	40 Gio	400 Go	0	équipe web	
`dev-data`	`horsprod`	6	24	96 Gio	2 000 Go	0	équipe data	
`socle`	—	6	16	32 Gio	500 Go	2	équipe plateforme	—
Total alloué		**82**	**380**	**1 128 Gio**	**22 700 Go**	**8**	à rapprocher de la capacité réelle du cloud et de la plage d'adresses externes	

Lecture : les nœuds `prod` et `horsprod` ont des quotas nuls — on n'y crée rien. Les bases n'ont aucune adresse flottante : elles ne sont pas exposables.

Les développeurs lisent la production sans pouvoir y écrire ; l'exploitation lit le hors-production. Personne n'écrit des deux côtés.

Chiffres illustratifs, construits pour l'exemple : à remplacer par l'inventaire réel et la méthode de dimensionnement vue plus haut.

05

Mise en œuvre sur Sunbeam

Sunbeam installe et exploite le cloud ; l'organisation en projets, rôles et quotas se construit ensuite par l'API d'OpenStack, avec les identifiants d'administration qu'il fournit.

- Pas à pas : l'arbre et les rôles
- Revue et suivi de consommation
- Synthèse
- Pas à pas : quotas, réseau, vérification
- Bonnes pratiques et erreurs fréquentes

Pas à pas (1) : l'arbre et les rôles

```
# Étape 0 — prérequis : cloud Sunbeam déployé,
# client openstack installé, domaine « nord » créé
sunbeam openrc > admin-openrc
source admin-openrc
openstack domain show nord -c id -c enabled

# Étape 1 — les nœuds de l'arbre
for n in prod horsprod; do
  openstack project create $n --domain nord \
    --description "Branche $n" --tag role:noeud
done
openstack project create socle --domain nord \
  --tag env:commun

# Étape 2 — les feuilles : projet:parent:env:application
for f in prod-web:prod:prod:web prod-bases:prod:prod:bases \
  prod-data:prod:prod:data rec-web:horsprod:rec:web \
  dev-web:horsprod:dev:web dev-data:horsprod:dev:data; do
  IFS=: read p par env app <<< "$f"
  openstack project create $p --domain nord \
    --parent $par --tag env:$env --tag app:$app
done
```

```
# Étape 3 — les groupes
for g in exploitation-prod audit developpeurs \
         equipe-web equipe-data dba; do
  openstack group create $g --domain nord
done

# Étape 4 — rôles hérités sur les nœuds : groupe:rôle:nœud
for a in exploitation-prod:member:prod developpeurs:reader:prod \
  audit:reader:prod audit:reader:horsprod \
  exploitation-prod:reader:horsprod; do
  IFS=: read g r n <<< "$a"
  openstack role add $r --inherited --group $g \
    --group-domain nord --project $n --project-domain nord
done

# Étape 5 — rôles directs sur les feuilles : groupe:projet
for a in equipe-web:rec-web equipe-web:dev-web \
  equipe-data:dev-data dba:prod-bases; do
  IFS=: read g p <<< "$a"
  openstack role add member --group $g --group-domain nord \
    --project $p --project-domain nord
done

# Contrôle : l'arbre, puis les droits effectifs
openstack project show prod --domain nord --children
openstack role assignment list --effective \
  --project prod-web --project-domain nord --names
```

> **Si le domaine est raccordé à un annuaire** (`sunbeam ldap add-domain`), les groupes viennent de l'annuaire : on saute l'étape 3 et l'on affecte les rôles aux groupes existants. Les automates de prod-web et prod-data reçoivent un identifiant d'application (module 8), pas un groupe.

Pas à pas (2) : quotas, réseau, vérification

```
# Étape 6 — quotas nuls sur les nœuds
for n in prod horsprod; do
  ID=$(openstack project show $n --domain nord -f value -c id)
  openstack quota set $ID --instances 0 --cores 0 \
    --ram 0 --volumes 0 --gigabytes 0 \
    --networks 0 --routers 0 --floating-ips 0
done

# Étape 7 — quotas des feuilles
ID=$(openstack project show prod-web --domain nord -f value -c id)
openstack quota set $ID \
  --instances 30 --cores 120 --ram 245760 \
  --volumes 60 --gigabytes 3000 --snapshots 60 \
  --networks 4 --subnets 8 --routers 1 --ports 100 \
  --floating-ips 4 --secgroups 20 --secgroup-rules 200

ID=$(openstack project show prod-bases --domain nord -f value -c id)
openstack quota set $ID \
  --instances 8 --cores 64 --ram 262144 \
  --volumes 32 --gigabytes 6000 \
  --networks 2 --routers 1 --ports 40 --floating-ips 0
```

> Même schéma pour chaque feuille : les valeurs viennent du tableau de dimensionnement, jamais d'une copie du projet voisin.

```
# Étape 8 — réseau commun du socle, partagé au cas par cas
openstack network rbac create --type network \
  --action access_as_shared \
  --target-project prod-web  commun

# Étape 9 — vérifier les plafonds posés
openstack quota show --usage $ID

# Étape 10 — vérifier la cloison avec le compte de dana,
# membre de developpeurs et d'equipe-web (pas d'admin-openrc)
export OS_USERNAME=dana OS_USER_DOMAIN_NAME=nord \
  OS_PROJECT_DOMAIN_NAME=nord
read -s OS_PASSWORD && export OS_PASSWORD
openstack --os-project-name prod-web \
  server create --flavor m1.small --image ubuntu essai
# attendu : 403 — la politique refuse la création
openstack --os-project-name dev-web server list
# attendu : uniquement les instances de dev-web
```

Essai	Résultat attendu
Développeur : lister en production	autorisé — `reader` hérité
Développeur : créer en production	refusé par la politique
Créer une instance dans le nœud `prod`	refusé par le quota
Adresse flottante dans `prod-bases`	refusée par le quota
31e instance dans `prod-web`	refusée par le quota

Revue et suivi de consommation

```
# Consommation et plafonds de chaque projet de production
for p in $(openstack project list --tags env:prod \
            -f value -c ID); do
  openstack quota show --usage $p
done

# Heures consommées sur une période, par projet
openstack usage list --start 2026-09-01 --end 2026-10-01

# Capacité réelle, côté hyperviseurs
openstack hypervisor list --long

# Qui a quoi : toutes les affectations d'une branche
openstack role assignment list --names \
  --project prod --project-domain nord --inherited
```

Ces relevés se prêtent à un script planifié : un tableau par mois suffit à repérer les dérives avant qu'elles ne deviennent des incidents.

Constat	Décision
Consommation > 80 % du quota	**relever avant le blocage, ou faire le ménage**
Consommation < 30 % depuis trois mois	**abaisser : la marge inutile se reprend**
Somme des quotas ≫ capacité	**surallocation à assumer par écrit, ou à réduire**
Projet sans étiquette de propriétaire	**retrouver le responsable, ou désactiver**
Projet sans ressource ni connexion	**désactiver, puis supprimer après délai**
Rôle direct sur un nœud	**vérifier qu'il est voulu : il ne sert qu'au nœud**
Utilisateur nommé plutôt qu'un groupe	**remplacer par une appartenance à un groupe**

- **Fréquence :** consommation chaque mois ; droits chaque trimestre ; modèle d'organisation chaque année.
- **Deux regards :** la plateforme vérifie la capacité, le propriétaire du projet confirme les droits.
- **Avant de supprimer un projet :** vider ses ressources dans chaque service — Keystone ne le fait pas.

5 · MISE EN ŒUVRE

Bonnes pratiques et erreurs fréquentes

À faire

- **Dessiner l'arbre avant de créer :** le parent est définitif.
- **Un projet par application et par environnement,** même pour une petite application.
- **Baisser les quotas par défaut** avant d'ouvrir le cloud : un projet neuf ne doit rien pouvoir consommer d'important.
- **Affecter les rôles à des groupes,** jamais à des personnes.
- **Réserver l'héritage aux populations transversales ;** les équipes applicatives reçoivent des rôles directs.
- **Décrire l'organisation en code** — Terraform ou scripts versionnés : l'arbre, les rôles et les quotas se relisent et se rejouent.
- **Tester la cloison** avec un compte ordinaire après chaque changement.
- **Garder un projet de démonstration** à part : celui que crée `sunbeam configure` n'a rien à faire en production.

Erreur fréquente	Conséquence
Tout dans un seul projet par équipe	**un essai épuise le quota de la production**
Travailler dans le projet `admin`	**ressources mêlées aux pouvoirs d'administration**
Croire que le parent voit ses enfants	**droits manquants, ou accordés trop haut pour compenser**
Rôle hérité posé sur le domaine « pour simplifier »	**tout nouveau projet est ouvert d'office**
Quotas illimités « provisoires »	**plus aucune protection ; jamais rétablis**
Réseau créé avec `--share`	**tous les projets y accèdent**
Arbre à quatre ou cinq niveaux	**droits effectifs illisibles ; limites hiérarchiques inutilisables**

> **Règle d'or :** chaque projet a un propriétaire, un usage et un plafond connus.

SYNTHÈSE

Ce qu'il faut retenir

Le projet et l'arbre

- Toute ressource appartient à un projet ; l'isolation est faite par chaque service.
- L'arbre range et fait descendre des rôles ; il ne donne aucune visibilité du parent sur l'enfant.
- Parent définitif, noms uniques dans le domaine.
- Rôle hérité : s'applique en dessous, pas au nœud.

Les quotas

- Un plafond par projet, par ressource, par service.
- Vérifiés à l'entrée de l'API ; ils ne réservent rien.
- Les valeurs par défaut sont le réglage le plus important.
- Limites unifiées : valeurs dans Keystone, modèle plat ou à deux niveaux.

L'organisation

- Quatre niveaux : domaine, projet, réseau, matériel.
- Croiser application et environnement.
- Arbre par environnement ou par application, selon qui a besoin du droit transversal.
- Un socle pour le commun ; rien d'écrit des deux côtés de la cloison.

Question	Réponse courte
Quand créer un projet ?	**dès qu'une application, un environnement ou un niveau de sensibilité diffère**
Quand créer un sous-projet ?	quand un même droit doit couvrir plusieurs projets, ou qu'un plafond commun doit les coiffer
Comment fixer un quota ?	besoin en pointe, plus 20 à 30 % ; cohérent entre calcul, stockage et réseau ; revu chaque mois
Sur Sunbeam ?	`sunbeam openrc` pour les identifiants d'administration ; tout le reste par l'API d'OpenStack, de préférence en code versionné

Sources : documentation de Keystone (limites unifiées, projets hiérarchiques), documentation de Nova (quotas, limites unifiées), documentation de Canonical OpenStack 2026.1 — consultées en octobre 2026. Les valeurs par défaut des quotas et les chiffres des exemples sont à vérifier sur votre déploiement.